

PRÁCTICA N.º 6 — HOJA DE EVIDENCIAS

Mensajería asíncrona con RabbitMQ

COM-600 · Microservicios · Gestión 2/2026 · Tema 4 — Comunicación entre Microservicios

Nombre y apellidos	David Franz Soliz Ortega
Código universitario (CU)	35-5261
Carrera y paralelo	Ingeniería de sistemas
Fecha de entrega	20/09/2026
Sistema operativo, versión de Node, de Docker y de la imagen de RabbitMQ	Windows 10 64 bits, Docker v29.7.2,node v24.18.1
Repositorio (URL)	https://github.com/David-Franz-Soliz-Ortega/MicroServiciosDFSO/tree/d753c873956cd6710927f66bf826c19948ccce40/practicass/practica%206

Cómo se llena: Este documento se completa mientras se ejecuta la Parte 1 del enunciado. Cada captura y cada pregunta lleva el mismo número aquí y allá.

Las capturas deben ser de su propia máquina: debe verse el nombre de usuario o del equipo en la terminal. Recorte solo lo necesario, pero que el comando y su salida se lean con claridad.

Al terminar, exporte este archivo a PDF y súbalo a la plataforma junto con el enlace del repositorio.

Control de avance

Marque con una X a medida que complete cada laboratorio.

Laboratorio	Capturas	Preguntas	Completado
Laboratorio 0 — El broker en marcha y su consola de administración	1, 2, 3	1	x
Laboratorio 1 — Anatomía del enrutamiento: exchange, cola y binding	4, 5, 6	2	x
Laboratorio 2 — El productor: publicar un evento cuando la operación se confirma	7, 8	3	x
Laboratorio 3 — El consumidor y la demostración del desacoplamiento	9, 10, 11	4	x
Laboratorio 4 — Exchange directo frente a exchange temático	12, 13	5	x
Laboratorio 5 — Confirmaciones y durabilidad: qué se pierde y qué se salva	14, 15, 16	6	x
Laboratorio 6 — Reintentos limitados y cola de mensajes muertos	17, 18	7	x
Laboratorio 7 — Idempotencia: el mismo evento dos veces sin doble efecto	19, 20	8	x

PARTE 1 — EVIDENCIAS DE LOS LABORATORIOS

Laboratorio 0 — El broker en marcha y su consola de administración

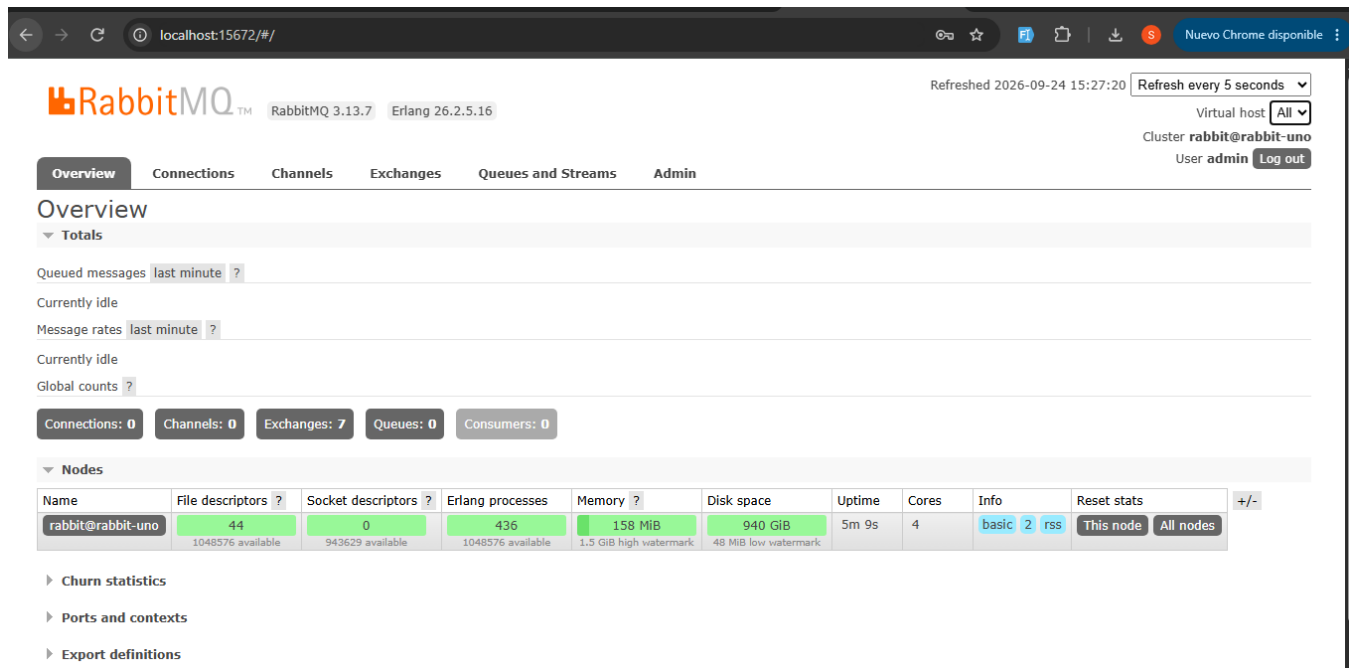
Captura N.º 1: salida de los cinco comandos anteriores en su terminal.

```
PS F:\2-2026\microservicios com600\practicas\practica > node --version
v24.18.1
PS F:\2-2026\microservicios com600\practicas\practica > docker --version
Docker version 29.7.2, build a7dcaa6
PS F:\2-2026\microservicios com600\practicas\practica > docker compose version
Docker Compose version v5.4.0
PS F:\2-2026\microservicios com600\practicas\practica > curl --version
curl 8.13.0 (Windows) libcurl/8.13.0 Schannel zlib/1.3.1 WinIDN
Release-Date: 2025-04-02
Protocols: dict file ftp ftps http https imap imaps ipfs ipns mqtt pop3 pop3s smb smbs smtp smtps telnet tftp ws wss
Features: alt-svc AsynchDNS HSTS HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM SPNEGO SSL SSPI threadsafe Unicode UnixSockets
PS F:\2-2026\microservicios com600\practicas\practica > docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
PS F:\2-2026\microservicios com600\practicas\practica > node --version
v24.18.1
PS F:\2-2026\microservicios com600\practicas\practica > 
```

Captura N.º 2: salida de docker compose ps con el contenedor com600-rabbit en estado running y las últimas líneas del registro con «Server startup complete».

```
PS F:\2-2026\microservicios com600\practicas\practica > docker compose logs --tail 5 rabbitmq
com600-rabbit | 2026-09-24 19:22:28.079631+00:00 [info] <0.680.0> * rabbitmq_federation
com600-rabbit | 2026-09-24 19:22:28.079631+00:00 [info] <0.680.0> * rabbitmq_management
com600-rabbit | 2026-09-24 19:22:28.079631+00:00 [info] <0.680.0> * rabbitmq_management_agent
com600-rabbit | 2026-09-24 19:22:28.079631+00:00 [info] <0.680.0> * rabbitmq_web_dispatch
com600-rabbit | 2026-09-24 19:22:28.155491+00:00 [info] <0.9.0> Time to start RabbitMQ: 22229 ms
PS F:\2-2026\microservicios com600\practicas\practica > docker compose ps
NAME                IMAGE              COMMAND              SERVICE   CREATED      STATUS      PORTS
com600-rabbit       rabbitmq:3-management "docker-entrypoint.s..." rabbitmq   2 minutes ago Up 2 minutes 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
PS F:\2-2026\microservicios com600\practicas\practica > 
```

Captura N.º 3: consola de administración de RabbitMQ abierta en la pestaña Overview, con el nombre del nodo rabbit-uno visible.



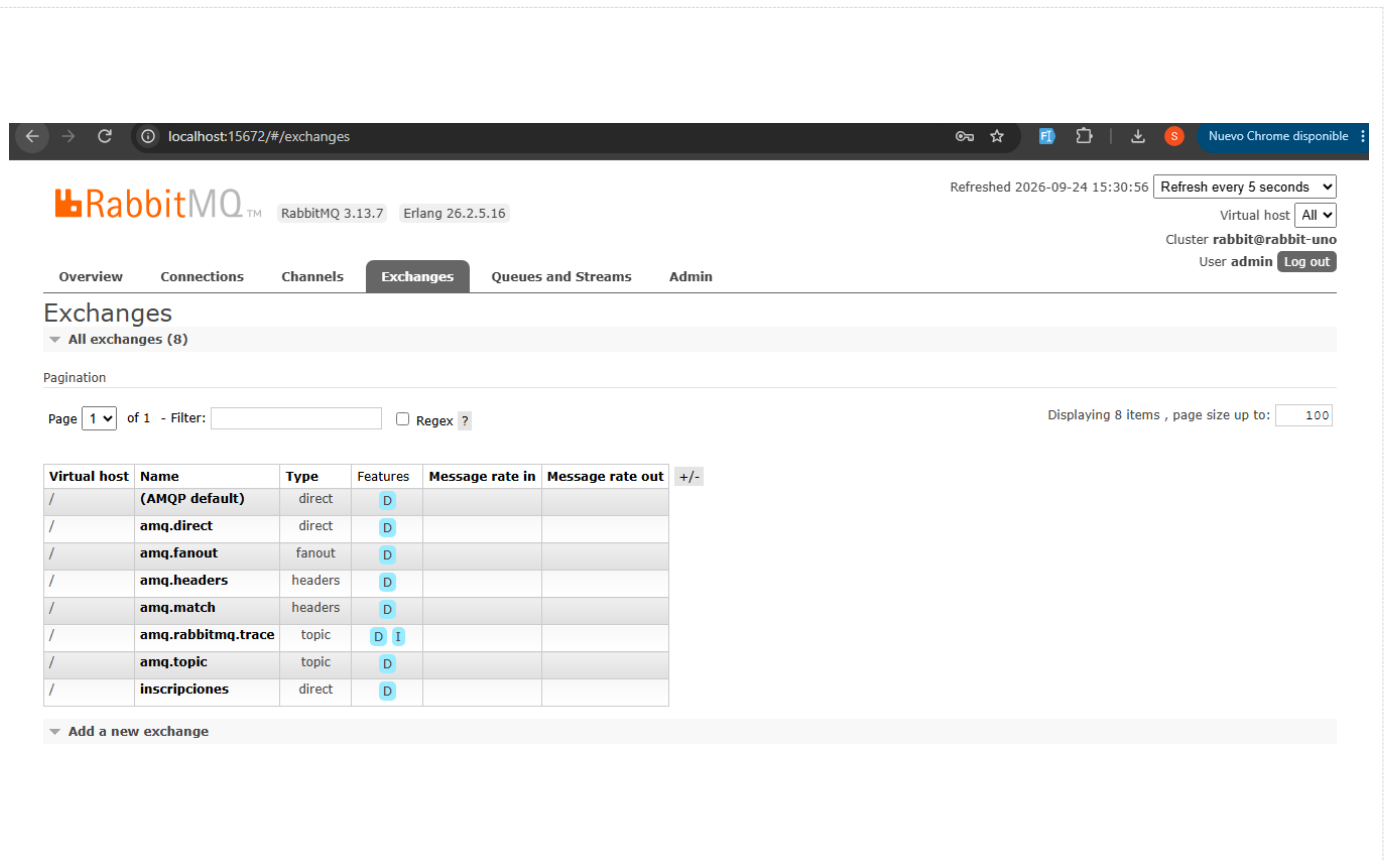
The screenshot shows the RabbitMQ Admin UI. At the top, there's a navigation bar with tabs: Overview, Connections, Channels, Exchanges, Queues and Streams, and Admin. The Overview tab is selected. Below the navigation bar, there's a 'Totals' section with metrics: Queued messages (last minute), Currently idle, Message rates (last minute), and Global counts. The Global counts section shows: Connections: 0, Channels: 0, Exchanges: 7, Queues: 0, Consumers: 0. Below this is a 'Nodes' section with a table showing the status of the node 'rabbit@rabbit-uno'. The table has columns: Name, File descriptors, Socket descriptors, Erlang processes, Memory, Disk space, Uptime, Cores, Info, and Reset stats. The node 'rabbit@rabbit-uno' has 44 file descriptors, 0 socket descriptors, 436 Erlang processes, 158 MiB memory, 940 GiB disk space, 5m 9s uptime, 4 cores, and is using the 'basic' plugin. Below the table, there are links for 'Churn statistics', 'Ports and contexts', and 'Export definitions'.

Pregunta N.º 1. En las prácticas 3, 4 y 5 el cliente hablaba directamente con el servicio y esperaba la respuesta. Aquí aparece un tercer proceso en el medio. ¿Qué gana el sistema al introducirlo y qué problema nuevo se crea con él?

Ganamos desacoplamiento y tolerancia a fallos: si el servicio de correos se cae, el servicio de inscripciones sigue funcionando y no se bloquea. El problema nuevo es que agregamos complejidad a la arquitectura.

Laboratorio 1 — Anatomía del enrutamiento: exchange, cola y binding

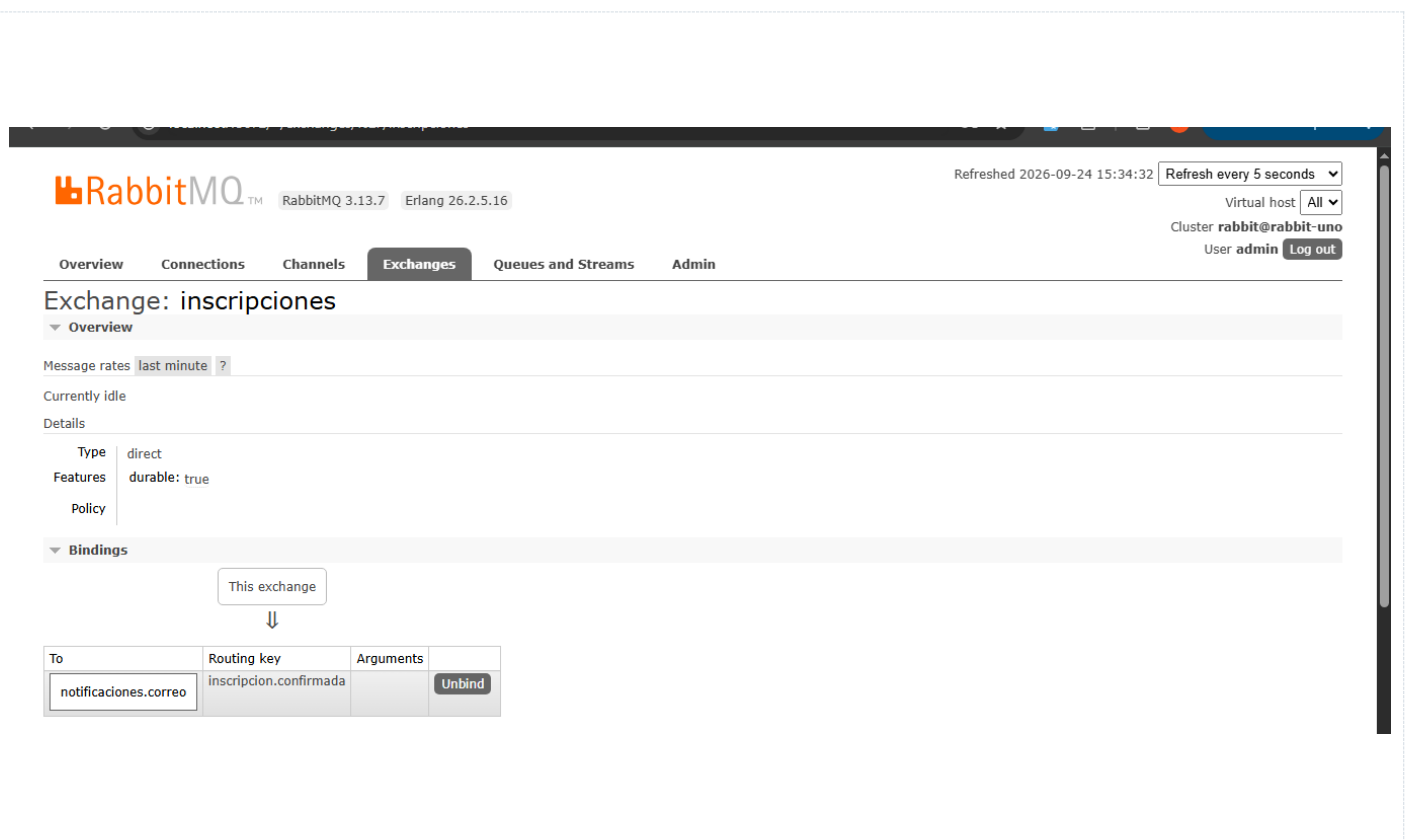
Captura N.º 4: lista de exchanges de la consola con el exchange inscripciones recién creado, mostrando su tipo direct y la marca D de durable.



The screenshot shows the RabbitMQ web interface at localhost:15672/#/exchanges. The 'Exchanges' tab is selected. A table lists the following exchanges:

Virtual host	Name	Type	Features	Message rate in	Message rate out	+/-
/	(AMQP default)	direct	D			
/	amq.direct	direct	D			
/	amq.fanout	fanout	D			
/	amq.headers	headers	D			
/	amq.match	headers	D			
/	amq.rabbitmq.trace	topic	D I			
/	amq.topic	topic	D			
/	inscripciones	direct	D			

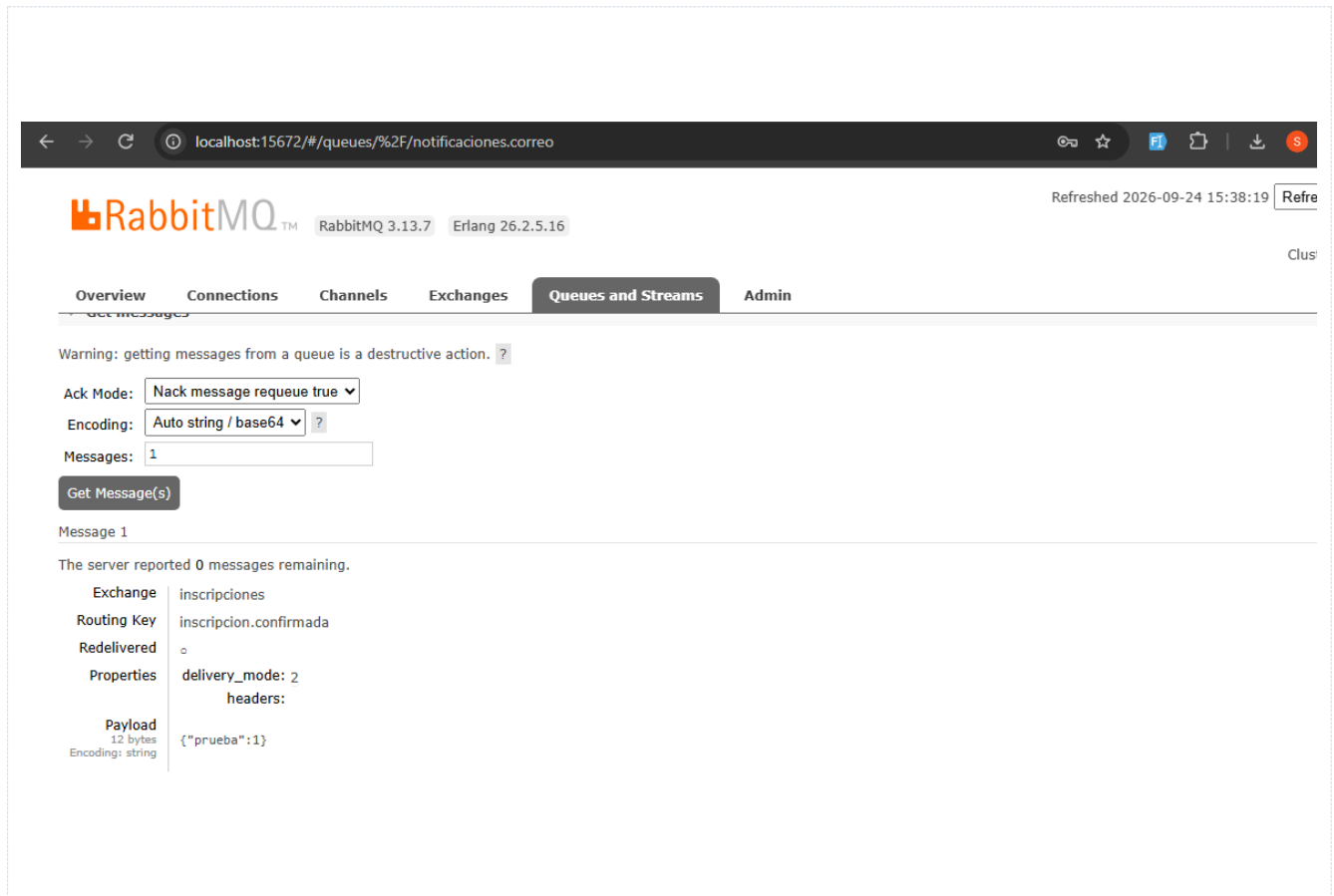
Captura N.º 5: página del exchange inscripciones mostrando el binding hacia la cola notificaciones.correo con la clave inscripcion.confirmada.



The screenshot shows the 'Exchange: inscripciones' page. Under the 'Bindings' section, a binding is shown to the 'notificaciones.correo' queue with the routing key 'inscripcion.confirmada'. The 'Unbind' button is visible next to the binding.

To	Routing key	Arguments
notificaciones.correo	inscripcion.confirmada	

Captura N.º 6: ventana Get messages mostrando el contenido del mensaje recuperado, con su clave de enrutamiento y sus propiedades.

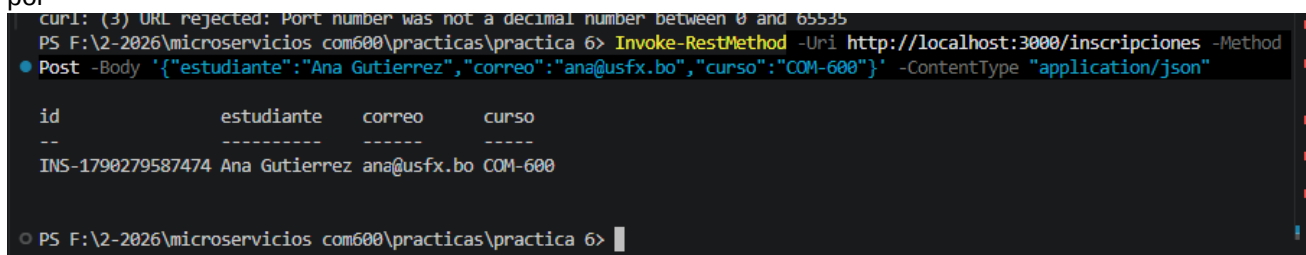


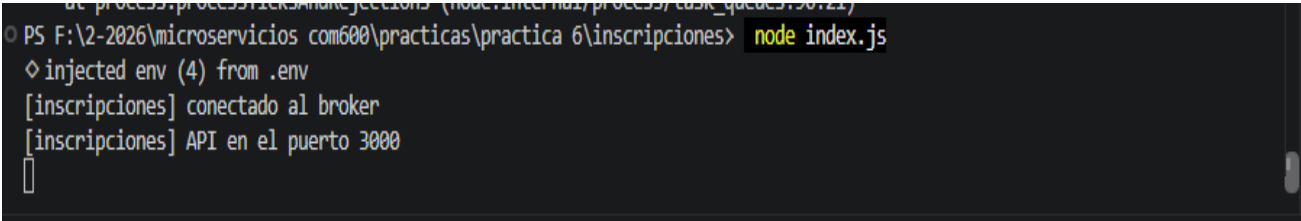
Pregunta N.º 2. El productor publica en el exchange y nunca nombra a la cola. Si mañana el área de finanzas también necesita enterarse de cada inscripción confirmada, ¿qué hay que modificar en el productor? Justifique su respuesta con lo que acaba de hacer a mano.

No hay que modificar absolutamente nada en el código del productor. Como el productor solo manda el mensaje al *exchange*, lo único que tendríamos que hacer es crear la nueva cola para Finanzas y vincularla al mismo exchange utilizando *inscripcion.confirmada*.

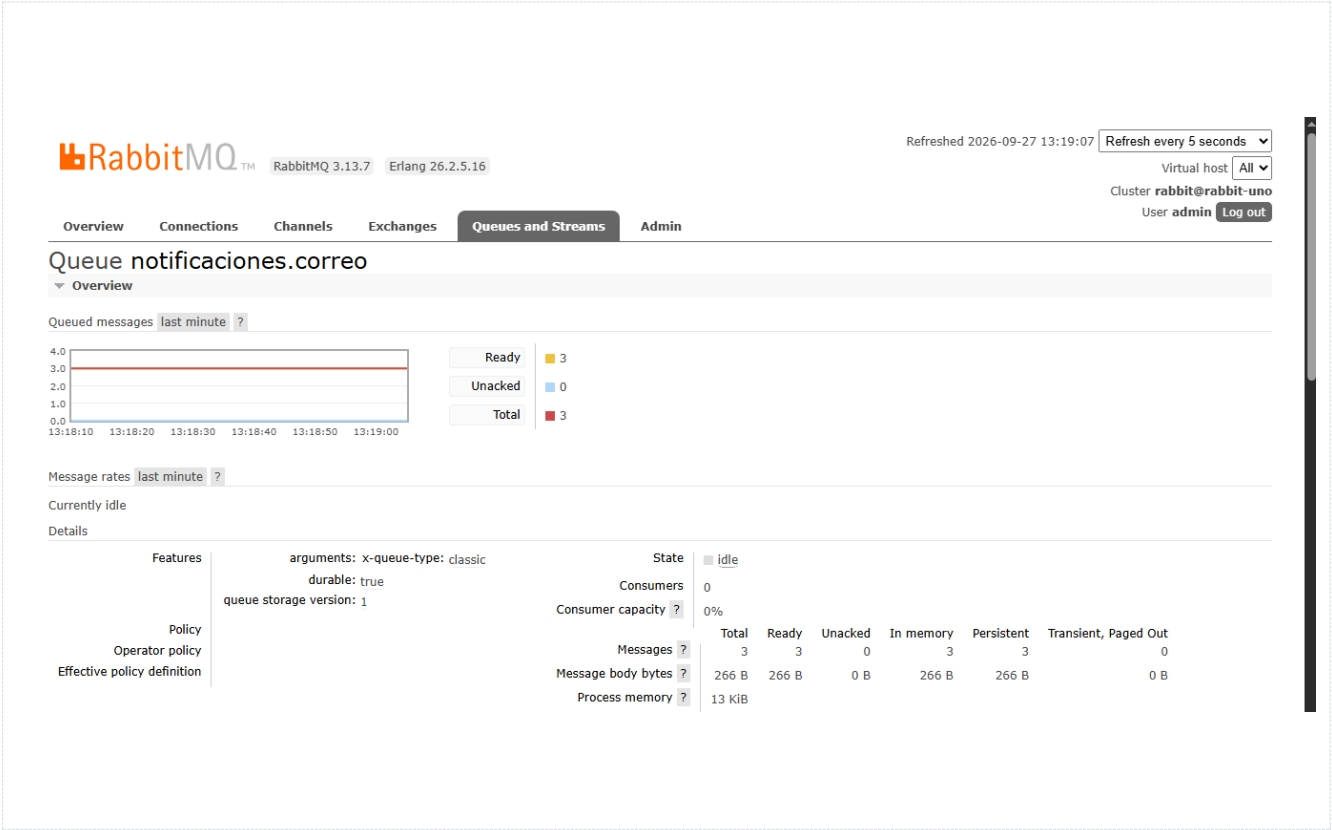
Laboratorio 2 — El productor: publicar un evento cuando la operación se confirma

Captura N.º 7: las dos terminales: el productor conectado al broker y la respuesta 201 con el registro devuelto por





Captura N.º 8: consola de administración mostrando la cola notificaciones.correo con un mensaje en la columna Ready y ningún consumidor.

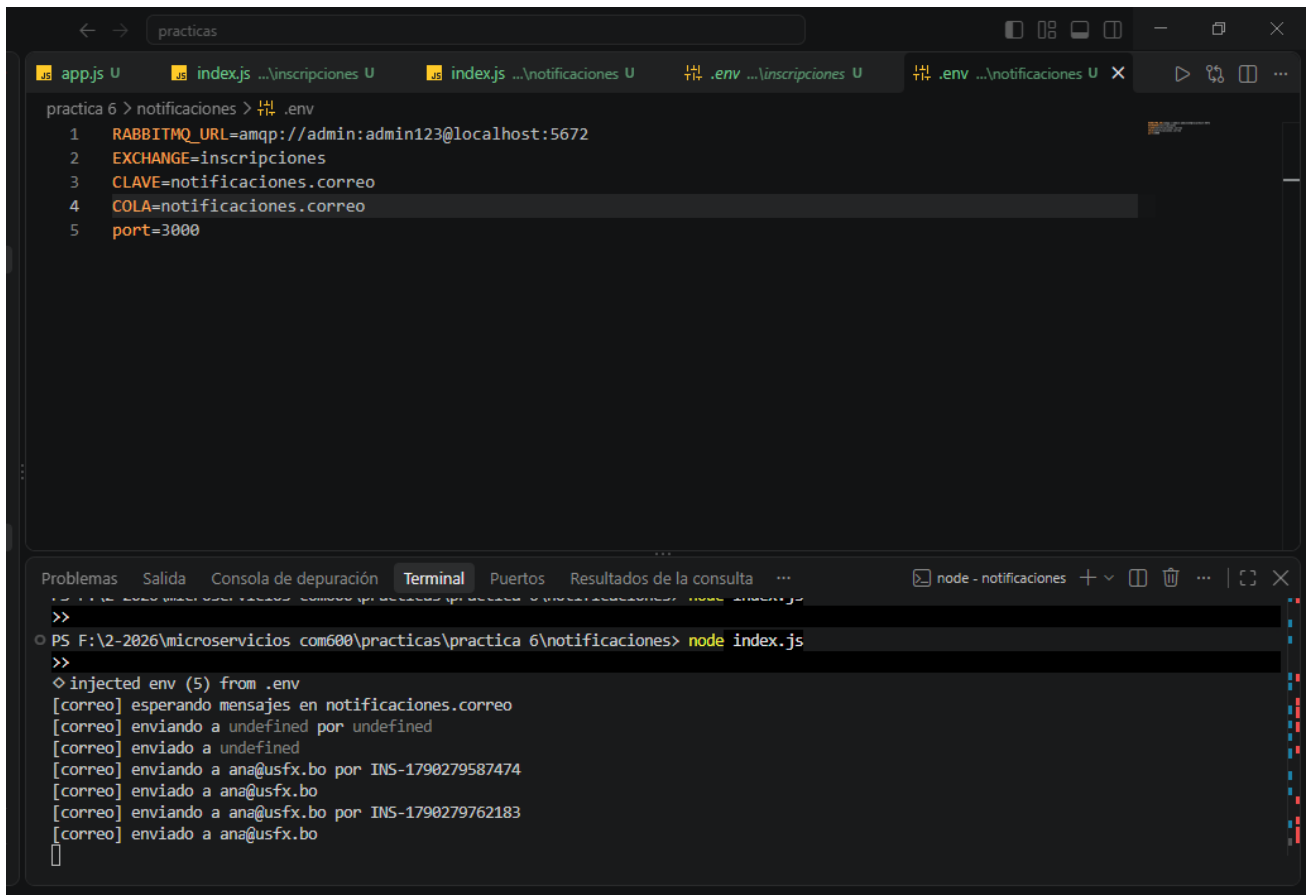


Pregunta N.º 3. La API respondió 201 aunque nadie haya enviado todavía el correo de bienvenida. ¿De qué trabajo se liberó el servicio de Inscripciones y qué promesa está haciendo a cambio al estudiante que se inscribió?

El servicio de Inscripciones se liberó de realizar la conexión de red pesada con el servidor y de esperar a que el correo salga, haciendo que la API responda casi instantáneamente.

Laboratorio 3 — El consumidor y la demostración del desacoplamiento

Captura N.º 9: terminal del consumidor procesando los eventos, con el registro de cada correo enviado y su identificador de inscripción.



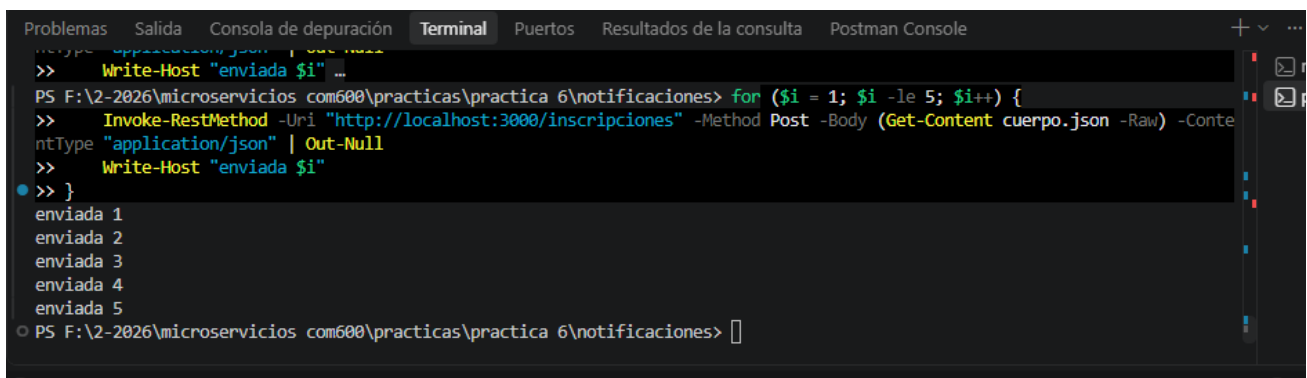
The screenshot shows a Visual Studio Code editor with a file explorer on the left and a terminal window at the bottom. The file explorer shows a project named 'practicas' with a subdirectory 'notificaciones'. The 'notificaciones' directory contains an '.env' file and an 'index.js' file. The '.env' file contains the following configuration:

```
1 RABBITMQ_URL=amqp://admin:admin123@localhost:5672
2 EXCHANGE=inscripciones
3 CLAVE=notificaciones.correo
4 COLA=notificaciones.correo
5 port=3000
```

The terminal window shows the command 'node index.js' being executed. The output indicates that the environment variables were injected and the application is running. The terminal output is as follows:

```
>>
PS F:\2-2026\microservicios com600\practicas\practica 6\notificaciones> node index.js
>>
◇ injected env (5) from .env
[correo] esperando mensajes en notificaciones.correo
[correo] enviando a undefined por undefined
[correo] enviado a undefined
[correo] enviando a ana@usfx.bo por INS-1790279587474
[correo] enviado a ana@usfx.bo
[correo] enviando a ana@usfx.bo por INS-1790279762183
[correo] enviado a ana@usfx.bo
[]
```

Captura N.º 10: consola de administración con la cola notificaciones.correo mostrando 5 mensajes en Ready, cero consumidores, y el gráfico de la cola creciendo.



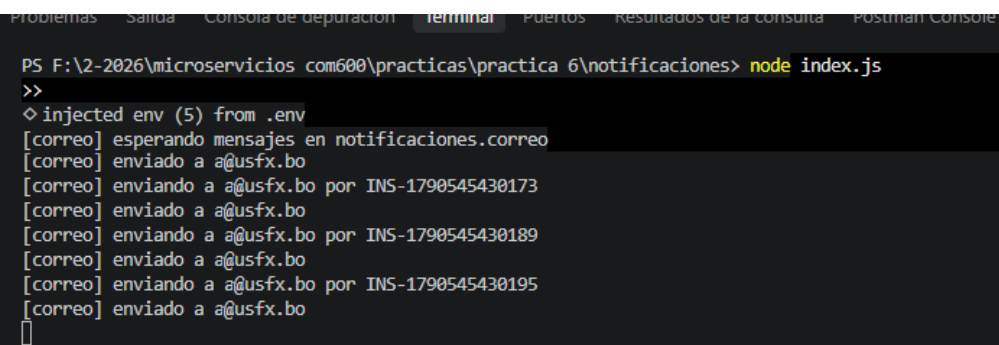
The screenshot shows a Visual Studio Code editor with a terminal window. The terminal window shows a script being executed that sends 5 messages to the 'inscripciones' queue. The script is as follows:

```
>> Write-Host "enviada $i" ...
PS F:\2-2026\microservicios com600\practicas\practica 6\notificaciones> for ($i = 1; $i -le 5; $i++) {
>> Invoke-RestMethod -Uri "http://localhost:3000/inscripciones" -Method Post -Body (Get-Content cuerpo.json -Raw) -Conte
ntType "application/json" | Out-Null
>> Write-Host "enviada $i"
}
>> }
enviada 1
enviada 2
enviada 3
enviada 4
enviada 5
PS F:\2-2026\microservicios com600\practicas\practica 6\notificaciones> []
```

Pregunta N.º 4. Con REST, si el servicio de Notificaciones estaba caído, la inscripción fallaba o se quedaba esperando. Aquí no falló nada. ¿Dónde está en este momento el trabajo pendiente, qué recurso está consumiendo mientras espera y qué ocurriría si el consumidor no volviera nunca?

El trabajo pendiente está almacenado en la cola dentro del servidor de RabbitMQ. Mientras espera, está consumiendo recursos físicos del broker (memoria RAM y espacio en disco). Si el consumidor se apaga y no vuelve nunca, la cola seguirá acumulando mensajes indefinidamente hasta agotar la memoria o el almacenamiento del servidor.

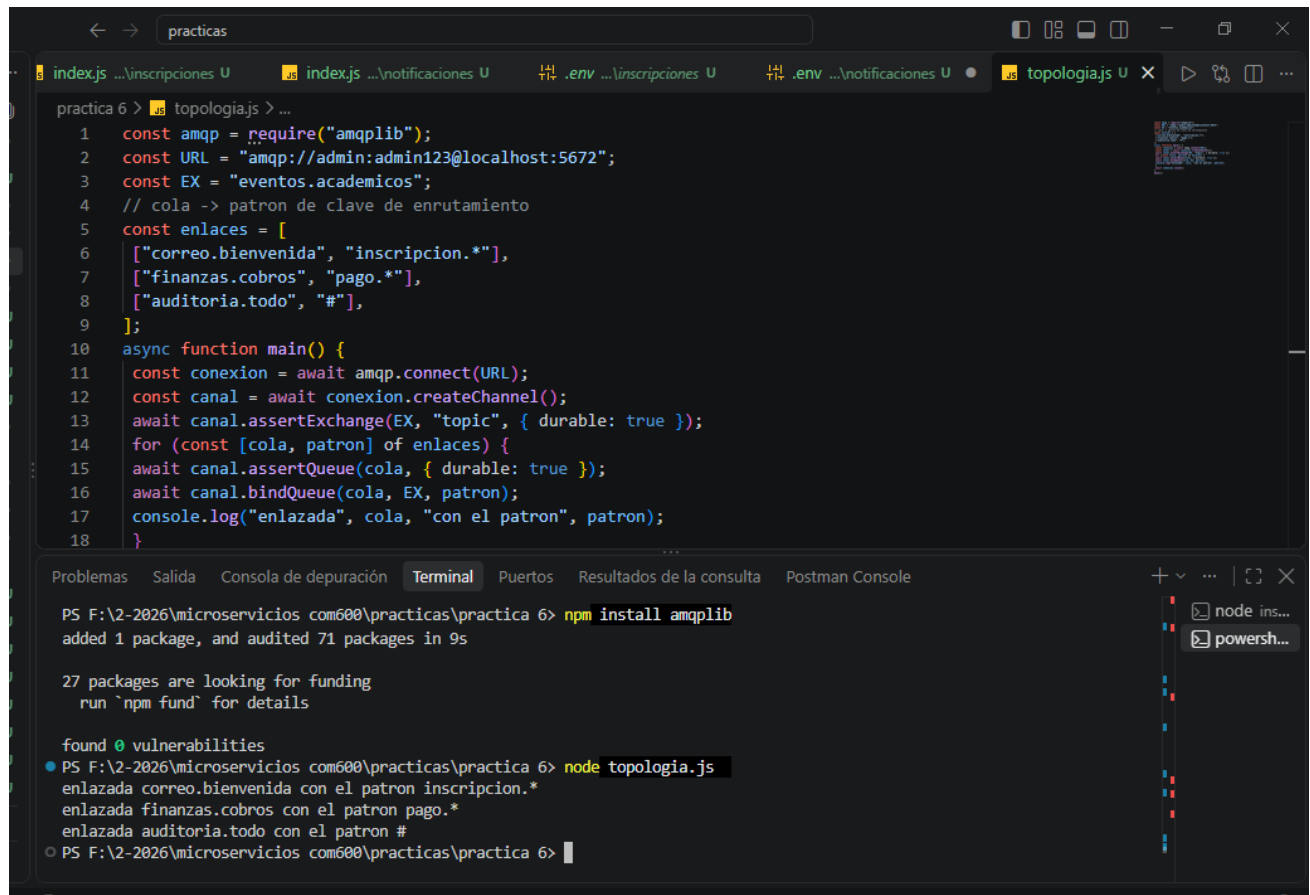
Captura N.º 11: la terminal del consumidor procesando los cinco mensajes acumulados y la cola de la consola bajando a cero en el gráfico.



```
PS F:\2-2026\microservicios com600\practicass\practica 6\notificaciones> node index.js
>>
◇ injected env (5) from .env
[correo] esperando mensajes en notificaciones.correo
[correo] enviado a a@usfx.bo
[correo] enviando a a@usfx.bo por INS-1790545430173
[correo] enviado a a@usfx.bo
[correo] enviando a a@usfx.bo por INS-1790545430189
[correo] enviado a a@usfx.bo
[correo] enviando a a@usfx.bo por INS-1790545430195
[correo] enviado a a@usfx.bo
█
```

Laboratorio 4 — Exchange directo frente a exchange temático

Captura N.º 12: página del exchange eventos.academicos en la consola, mostrando sus tres bindings con los patrones inscripcion.*, pago.* y #.



```
practica 6 > topologia.js > ...
1  const amqp = require("amqplib");
2  const URL = "amqp://admin:admin123@localhost:5672";
3  const EX = "eventos.academicos";
4  // cola -> patron de clave de enrutamiento
5  const enlaces = [
6    ["correo.bienvenida", "inscripcion.*"],
7    ["finanzas.cobros", "pago.*"],
8    ["auditoria.todo", "#"],
9  ];
10
11 async function main() {
12   const conexion = await amqp.connect(URL);
13   const canal = await conexion.createChannel();
14   await canal.assertExchange(EX, "topic", { durable: true });
15   for (const [cola, patron] of enlaces) {
16     await canal.assertQueue(colas, { durable: true });
17     await canal.bindQueue(colas, EX, patron);
18     console.log("enlazada", cola, "con el patron", patron);
19   }
20 }
```

Problemas Salida Consola de depuración Terminal Puertos Resultados de la consulta Postman Console

```
PS F:\2-2026\microservicios com600\practicas\practica 6> npm install amqplib
added 1 package, and audited 71 packages in 9s

27 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
● PS F:\2-2026\microservicios com600\practicas\practica 6> node topologia.js
enlazada correo.bienvenida con el patron inscripcion.*
enlazada finanzas.cobros con el patron pago.*
enlazada auditoria.todo con el patron #
○ PS F:\2-2026\microservicios com600\practicas\practica 6>
```

Captura N.º 13: lista de colas de la consola con los contadores de correo.bienvenida, finanzas.cobros y auditoria.todo después de publicar las cuatro claves.

localhost:15672/#/queues

Refreshed 2026-09-29 10:45:32

RabbitMQ 3.13.7 Erlang 26.2.5.16

Overview Connections Channels Exchanges **Queues and Streams** Admin

Queues

▼ All queues (7)

Pagination

Page 1 of 1 - Filter: ☐ Regex ?

Displaying 7 items

Overview					Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	amq.gen-5bcrKCnxHgJKeDC0AsGGzA	classic	D	running	0	0	0				
/	amq.gen-YzZQOVf1yOxbJeqwHx3jHA	classic	D	running	0	0	0				
/	auditoria.todo	classic	D	running	0	0	0				
/	correo.bienvenida	classic	D	running	0	0	0				
/	finanzas.cobros	classic	D	running	0	0	0				
/	inscripciones	classic	D	running	0	0	0				
/	notificaciones.correo	classic	D Args	running	0	0	0	0.00/s	0.00/s	0.00/s	

▼ Add a new queue

Virtual host: /

Pregunta N.º 5. La clave certificado.emitido llegó solo a una de las tres colas. Explique por qué, y diga qué diferencia hay entre el comodín * y el comodín # con un ejemplo propio de su proyecto integrador.

Llegó a una sola cola porque las demás tenían un binding que no hacía match con la ruta exacta de ese mensaje. La diferencia es que el asterisco (*) sustituye exactamente una palabra, mientras que el numeral (#) sustituye cero o más palabras.

Laboratorio 5 — Confirmaciones y durabilidad: qué se pierde y qué se salva

Captura N.º 14: consola mostrando la cola con el mensaje que pasó de Unacked a Ready tras la caída del consumidor.

```
PS F:\2-2026\microservicios com600\practicass\practica 6\notificaciones> node index.js
◇ injected env (5) from .env
[correo] esperando mensajes en notificaciones.correo
[correo] enviando a a@usfx.bo por INS-1790693348355
[correo] enviando a a@usfx.bo por INS-1790693642502
[correo] enviando a a@usfx.bo por INS-1790694571536
[correo] enviado a a@usfx.bo
[correo] enviado a a@usfx.bo
[correo] enviado a a@usfx.bo
[correo] enviando a a@usfx.bo por INS-1790694596873
[correo] enviado a a@usfx.bo
█
```

Connect Lín. 10, col. 26 Espacios: 4 UTF-8

Captura N.º 15: consola mostrando la cola en cero después de la caída del consumidor con noAck en true: el mensaje se perdió.

```
[correo] enviado a c@usfx.bo
PS F:\2-2026\microservicios com600\practicass\practica 6\notificaciones> node index.js
◇ injected env (5) from .env
[correo] esperando mensajes en notificaciones.correo
[correo] enviando a c@usfx.bo por INS-1790696356079
[correo] enviado a c@usfx.bo
█
```

Captura N.º 16: lista de colas después del reinicio del broker: la cola durable con sus mensajes intactos y la cola transitoria desaparecida.

```
PS F:\2-2026\microservicios com600\practicas\practica 6\notificaciones> node index.js
◇ injected env (5) from .env
[correo] esperando mensajes en notificaciones.correo
█

PS F:\2-2026\microservicios com600\practicas\practica 6\notificaciones> docker compose logs --tail=3 rabbitmq
com600-rabbit | 2026-09-29 15:41:13.075898+00:00 [info] <0.569.0> * rabbitmq_management_agent
com600-rabbit | 2026-09-29 15:41:13.075898+00:00 [info] <0.569.0> * rabbitmq_web_dispatch
com600-rabbit | 2026-09-29 15:41:13.134982+00:00 [info] <0.9.0> Time to start RabbitMQ: 29413 ms
PS F:\2-2026\microservicios com600\practicas\practica 6\notificaciones> █
```

Pregunta N.º 6. Se necesitan dos condiciones para que un mensaje sobreviva al reinicio del broker: la cola durable y el mensaje persistente. Explique qué pasa exactamente si se cumple solo una de las dos, y por qué la durabilidad tiene un costo que no siempre conviene pagar.

Si la cola es durable pero el mensaje no es persistente, al reiniciar RabbitMQ la cola seguirá existiendo pero estará vacía. Si el mensaje es persistente pero la cola no es durable, al reiniciar se elimina la cola y el mensaje se pierde con ella. Escribir mensajes en el disco físico de forma persistente tiene un costo de rendimiento alto (I/O) porque el disco es mucho más lento que la memoria RAM; no vale la pena pagar ese costo por mensajes triviales como logs temporales.

Laboratorio 6 — Reintentos limitados y cola de mensajes muertos

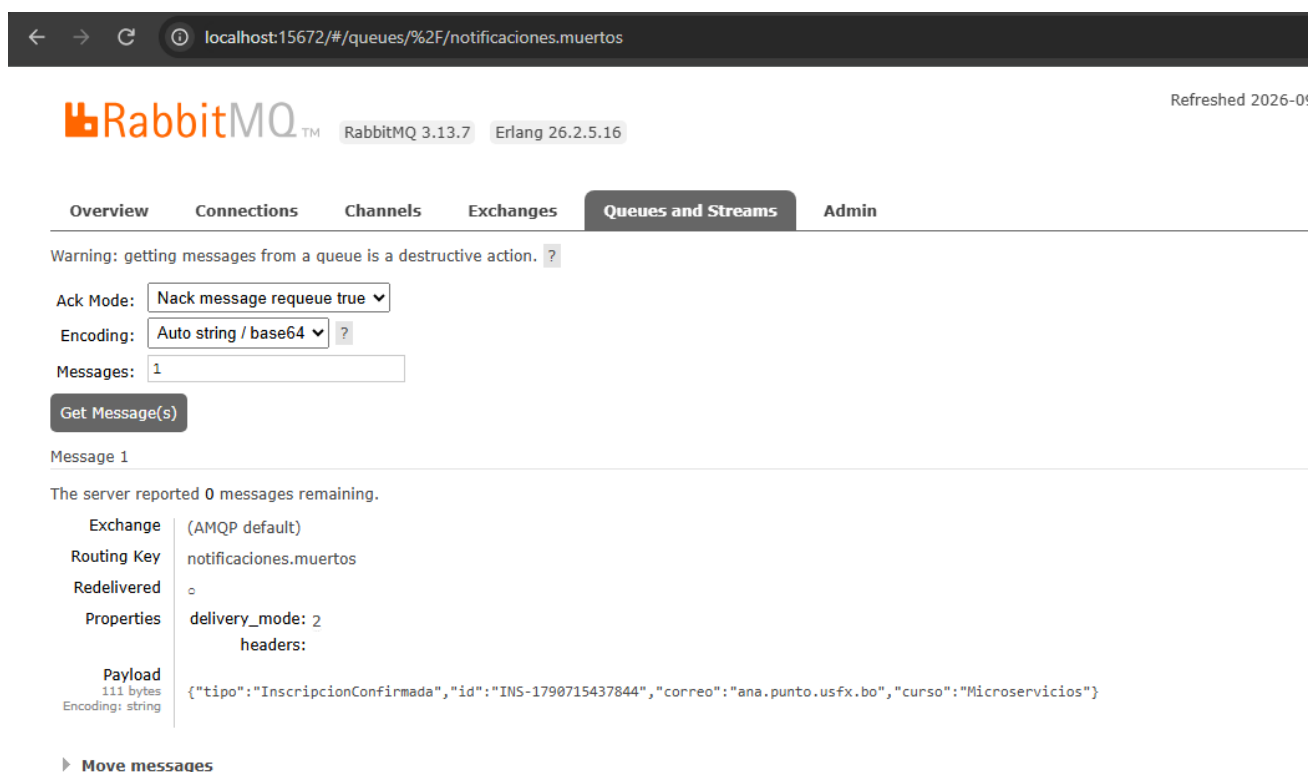
Captura N.º 17: terminal del consumidor mostrando los tres intentos numerados y fallidos del mismo mensaje, con cinco segundos entre uno y otro.

```
Problemas 2 Salida Consola de depuración Terminal Puertos Resultados de la consulta Postman Console

PS F:\2-2026\microservicios com600\practicas\practica 6\notificaciones> node index.js
}

Node.js v24.18.1
PS F:\2-2026\microservicios com600\practicas\practica 6\notificaciones> node index.js
◇ injected env (5) from .env
[correo] esperando mensajes en notificaciones.correo
[correo] intento 1 fallido: correo invalido: ana.punto.usfx.bo
[correo] intento 2 fallido: correo invalido: ana.punto.usfx.bo
[correo] intento 3 fallido: correo invalido: ana.punto.usfx.bo
[correo] sin mas intentos: a la cola muerta
█
```

Captura N.º 18: consola mostrando la cola notificaciones.muertos con el mensaje dentro y su contenido desplegado en Get messages.



localhost:15672/#/queues/%2F/notificaciones.muertos

Refreshed 2026-01-10 10:10:10

RabbitMQ 3.13.7 Erlang 26.2.5.16

Overview Connections Channels Exchanges **Queues and Streams** Admin

Warning: getting messages from a queue is a destructive action. ?

Ack Mode:

Encoding: ?

Messages:

Message 1

The server reported 0 messages remaining.

Exchange	(AMQP default)
Routing Key	notificaciones.muertos
Redelivered	0
Properties	delivery_mode: 2 headers:
Payload 111 bytes Encoding: string	{"tipo": "InscripcionConfirmada", "id": "INS-1790715437844", "correo": "ana.punto.usfx.bo", "curso": "Microservicios"}

► Move messages

Pregunta N.º 7. La cola de mensajes muertos no tiene ningún consumidor automático conectado. ¿Por qué sería un error ponerle uno que reintente sin más? Describa qué debería hacer, paso a paso, la persona o el proceso que atiende esa cola.

Sería un error porque generaríamos un bucle infinito de consumo de CPU. Si el mensaje llegó a la cola muerta, es porque ya superó sus intentos y tiene un error insalvable (como el JSON mal formado o el correo sin arroba) que volverá a hacer explotar al consumidor.

Laboratorio 7 — Idempotencia: el mismo evento dos veces sin doble efecto

Captura N.º 19: terminal del consumidor mostrando el mismo identificador de evento procesado dos veces y el cupo descontado dos veces.

```
[correo] intento 2 fallido: correo invalido: maria.punto.usfx.bo
[correo] sin mas intentos: a la cola muerta
[correo] intento 3 fallido: correo invalido: maria.punto.usfx.bo
[correo] sin mas intentos: a la cola muerta
[correo] bienvenida enviada a ana@usfx.bo
[cupos] COM-600 quedan 29
█
```

Captura N.º 20: terminal del consumidor mostrando el mensaje repetido reconocido e ignorado, con el cupo descontado una sola vez.

```
practica 6 > notificaciones > index.js > ...
22  async function main() {
34    canal.consume(COLA, (mensaje) => {
45      return canal.ack(mensaje);
46    }
47    try {
48      procesar(JSON.parse(mensaje.content.toString()));
49      canal.ack(mensaje);
50    } catch (e) {
51      console.error("[correo] intento", intentos + 1, "fallido:", e.message);
52      canal.nack(mensaje, false, false);
53    }
54  }, { noAck: false });
55 }
56
57 function procesar(evento) {
58   if (!evento.correo || !evento.correo.includes("@")) {
59     throw new Error("correo invalido: " + evento.correo);
60   }
61 }
```

```
PS F:\2-2026\microservicios com600\practicas\practica 6\notificaciones> node index.js
[cupos] COM-600 quedan 23
[correo] bienvenida enviada a ana@usfx.bo
[cupos] COM-600 quedan 22
[correo] bienvenida enviada a ana@usfx.bo
[cupos] COM-600 quedan 21
[correo] bienvenida enviada a ana@usfx.bo
[cupos] COM-600 quedan 20
[correo] bienvenida enviada a ana@usfx.bo
[cupos] COM-600 quedan 19
[correo] ignorando evento repetido: TEST-001
```

Pregunta N.º 8. El broker garantiza la entrega al menos una vez, pero no exactamente una vez. Explique por qué le resulta imposible garantizarlo y en qué parte del sistema queda entonces la responsabilidad. Dé un ejemplo de su proyecto integrador donde una entrega repetida causaría un daño visible al usuario.

Resulta imposible porque la red no es 100% confiable. Si el consumidor procesa el mensaje pero ocurre un microcorte de red o el servicio se reinicia justo antes de enviar el ack (confirmación) de vuelta, el broker no tiene forma de saber si el fallo ocurrió antes o después de aplicar el efecto. Ante la duda, el broker está obligado a reenviar el mensaje para cumplir su promesa de no perder datos.

Cierre de la Parte 1

Responda brevemente antes de entregar:

¿Qué parte de la práctica le resultó más difícil y por qué?

La inclusión de dlq.js y la parte final debido a errores que me salían al querer usar un cuerpo.json

¿Qué error tuvo que resolver por su cuenta y cómo lo resolvió?

La ejecución de dlq.js e index.js de notificaciones tuve que buscar como arreglar los errores de código que no me dejaban ejecutar node

¿Qué haría distinto si tuviera que repetir esta práctica?

Usar herramientas que me den lo mismo que usar curls como insomnia.

Declaración de autoría: Declaro que las capturas presentadas fueron tomadas en mi propio equipo y que las respuestas son de mi autoría. Entiendo que presentar evidencias de otra persona anula la Parte 1 de esta práctica.



Firma del estudiante

29/09/2026

Fecha